Week 13 · Refactoring & owning the code

Technical Interaction Design · ITU · Autumn 2026

Built 8 November 2026 · commit `55039c3`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-13.pdf

CONTENTS

Component Extraction Cheat Sheet

- Every pull request is judged against the same five questions
- Extract a block when it repeats, or when you can name it and the page reads better without it
- Four tests catch most bad extractions
- An extraction moves JSX, and whatever it used from outside becomes a prop
- The same mistakes show up in every review
- For a deeper dive

Component Extraction Cheat Sheet

Source: Lectures/Technical/Structure/Component-Extraction-Cheatsheet.md — something unclear? Open an issue, or send a pull request.

One page, for when you are reviewing a pull request, your own or a teammate's. The reasoning behind it is in Finding the Components, Pulling Out the Service Layer and Code Organization; the examples are all from `todo-26`.

Every pull request is judged against the same five questions

From this week on, this is the review checklist. Before you approve a pull request, you should be able to answer *yes* to all five:

1. **Small components.** Does each component do one job, with everything in it at the same level of abstraction?
2. **State where it belongs.** Is every piece of state in the lowest component that needs it, and no lower?
3. **Clear names.** Do components have nouns for names and handlers verbs, and does every name still fit every place it is used?
4. **Parse behind a service.** Does only `services/` import Parse?
5. **No needless dependencies.** Is every new package in `package.json` worth what it costs? (See Libraries.)

State where it belongs gets its full answer in week 12, with lifting state up. Until then: if two components need the same state, it lives in their closest common parent.

Something unclear in this section? Open an issue.

Extract a block when it repeats, or when you can name it and the page reads better without it

```
Is this JSX the same thing, drawn more than once?
├─ YES → extract it, and name what it draws                  (MyListRow)
└─ NO  → Can you name it in a word or two, as a thing?
         ├─ YES → Would the page read better with just the name?
         │        ├─ YES → extract it
         │        └─ NO  → leave it where it is
         └─ NO  → it is not one thing yet; leave it, or split it
```

When unsure, leave it. Extracting later takes five minutes; a component nobody needed stays forever.

Something unclear in this section? Open an issue.

Four tests catch most bad extractions

Test	Ask	In todo-26
Name	Can you name it as a thing, in a word or two?	`SharedListRow`, not `LeaveButtonAndStuff`
Halves	Would either half be useful on its own? Then it is two.	`AddForm`'s input and button: no, so one
Repeat	Is it drawn more than once?	`MyListRow`, once per list
Level	Is everything in the parent at the same level of abstraction?	`Home` after: headings, rows, a form; no `.get("owner")`

Something unclear in this section? Open an issue.

An extraction moves JSX, and whatever it used from outside becomes a prop

1. **Cut** the block of JSX out of the parent.
2. **List** everything it used and did not define itself: variables from a `map`, the parent's props, handlers. Each one becomes a prop.
3. **Leave the state** where it was, unless only the new component uses it.
4. **Leave the `key`** on the call site, in the `map`; it belongs to the list, not the row.
5. **Click through** the app. The behaviour must not change.

```
// before, in Home
{lists.map((item) => (
  <div key={item.id}>
    <Link to={`/lists/${item.id}`}>{item.get("name")}</Link>
    <AddForm onAdd={(username) => onShare(item, username)} buttonLabel="Share with
        (username)" />
  </div>
))}

// after: item and onShare were used from outside, so they are the props
{lists.map((list) => (
  <MyListRow key={list.id} list={list} onShare={onShare} />
))}
```

Something unclear in this section? Open an issue.

The same mistakes show up in every review

Mistake	Why it hurts	Instead
A component named for an action: `ShareList`	It names the button's job, not the thing on the screen	`MyListRow`, with `handleShare` inside
A name that stopped fitting: `NewTodoForm` used to share lists	The name lies to the next reader	Rename it when its job grows: `AddForm`
`useState` moved into an input wrapper	The parent can no longer read or clear the value	The parent keeps the state; the wrapper takes `value` and `onChange`
`onChange={onChange}` on a wrapped `<input>`	The parent gets an event where it expected text	`onChange={(event) ⇒ onChange(event.target.value)}`
A query in a click handler	The component knows how the database works	A function in `services/`, named in the app's words
`alert` in a service	The service decides how the UI reports errors	The service throws; the component catches and tells the user
The page's heading inside `ToDoList`	The list cannot be drawn without the title, nor the title with anything else	The page draws the heading; `ToDoList` draws the list
A component for every `<div>`	Clutter, for nothing	The stopping rule: extract when it repeats

Something unclear in this section? Open an issue.

For a deeper dive

- React Code Quality: a longer worked example, on a different app
- Semantic JSX: the argument for naming what a block *is*
- Thinking in React

Something unclear in this section? Open an issue.